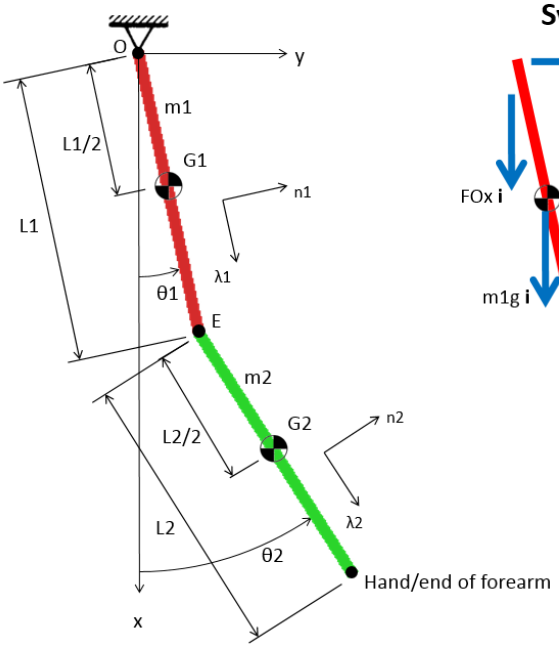
Author: Aaron Sandoval
Class: MAE 4730/5730
Instructor: Professor Andy Ruina
Written: October 25, 2017

Homework Solutions

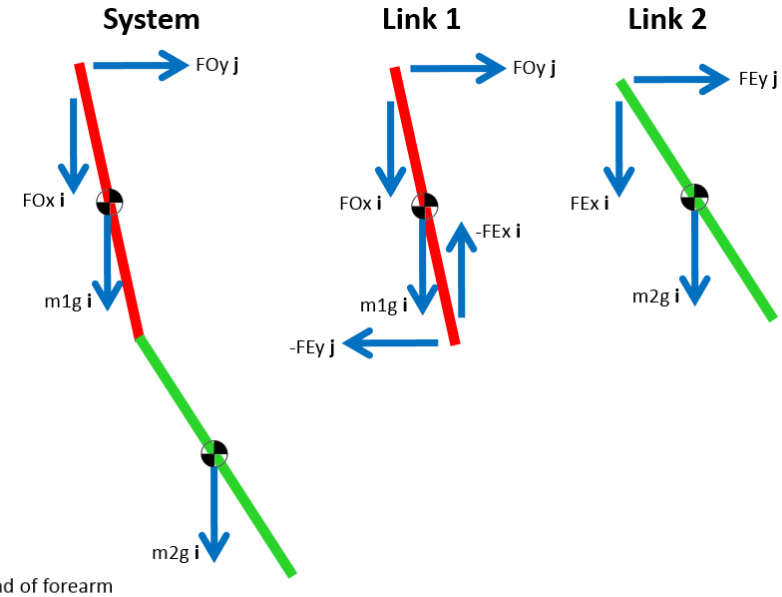
25. **Double pendulum**. Consider a double pendulum with 2 uniform bars of equal length L = 1, g = 1.

a. Solve using AMB. Assume the initial conditions: $\theta_1 = \frac{\pi}{2};\ \theta_2 = \pi; zero\ initial\ velocity$. Integrate until t = 30, and draw the trajectory of the end of the forearm.

Setup:



FBDs:



All calculations in MATLAB, see Appendix.

$AMB_{system/O}$, minimal coordinates
$AMB_{link\ 2/E}$, minimal coordinates
Produces 2 equations for 2 unknowns $\ddot{\theta}_1, \ddot{\theta}_2$ solved in RHS file.

This homework solution has been approved by Professor Ruina.

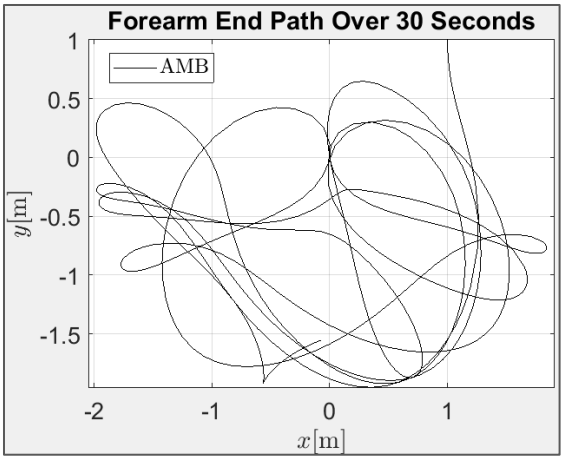


Figure 1: Hand path, AMB

The path appears chaotic. The point will never reach its initial height since all KE would be exchanged for PE, which is not possible in a finite time.

b. Solve again with DAEs, and show that the equations of motion are the same.

$LMB_{link\ 1}$, maximal coordinates	2 equations
$AMB_{link\ 1/O}$, maximal coordinates	1 equation
$LMB_{link\ 2}$, maximal coordinates	2 equations
$AMB_{link\ 2/E}$, maximal coordinates	1 equation
4 constraints	4 equations
Result:	10 equations for 10 unknowns

Derived symbolically, solved numerically in RHS file.

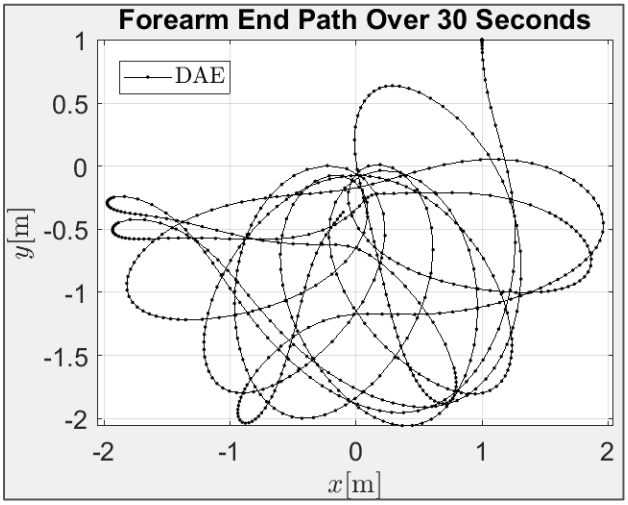


Figure 2: Forearm end path calculated using DAEs.

At a glance, the path appears nearly identical to that of the AMB solution. However after closer

examination, the differences emerge, and it is seen that the paths are completely diverged by 30 seconds. A few curves reach further down from the origin than 2 links of length 1 should reach, indicating numerical error accumulation.

c. And again with Lagrange equations, and compare again.

Similarly to the DAE approach, the Lagrange equations form the linear system of equations symbolically in the derivation script, which is exported to a RHS file to be solved numerically during execution.

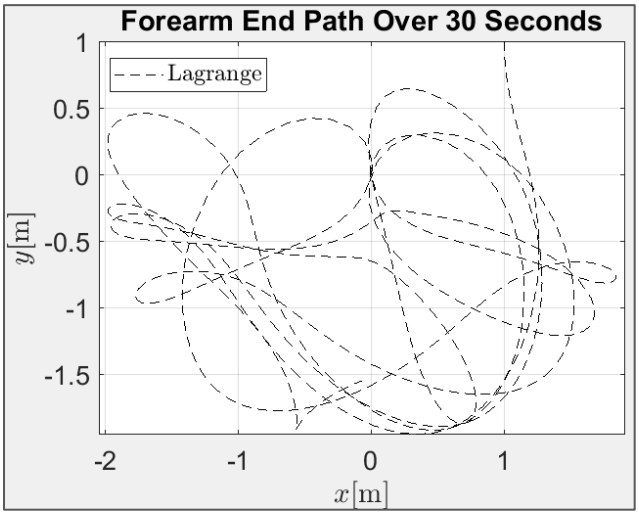


Figure 3: Forearm end path calculated using Lagrange equations.

The path is identical to the AMB path to the eye. The divergence is not visible on this scale. Figure 4 shows the ending positions, where it may be seen that the positional error is less than 0.00001 m.

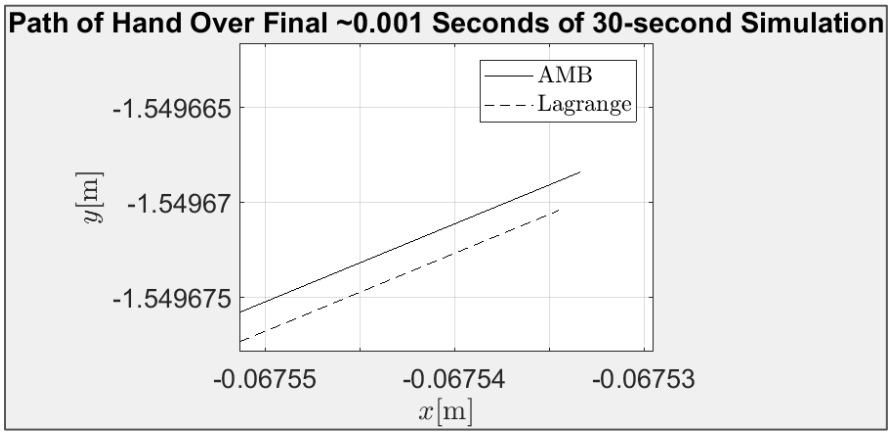


Figure 4: End of near-identical hand trajectories for AMB and Lagrange solutions.

Error Analysis:

We decide to measure the error of a solution as the difference in θ_2 over time between a given

path and the AMB path. Chaotic systems have exponential error propagation, so we expect to observe this type of growth in the error data.

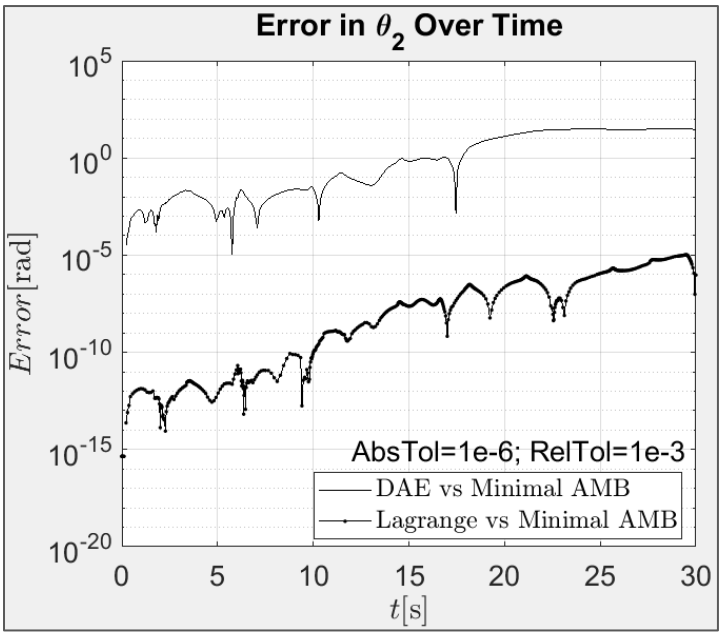


Figure 5: Growth in error over 30 seconds for DAE and Lagrange solutions.

The error growth is generally exponential (linear on this log plot), with expected chaotic oscillations superposed. This is consistent with expectations for a chaotic system. We now simulate again for a longer period.

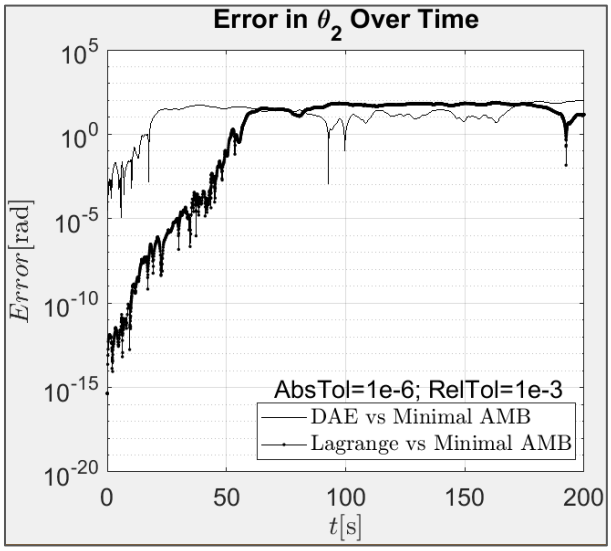


Figure 6: Growth in error over 200 seconds for DAE and Lagrange solutions.

The exponential growth abruptly stops near 10 radians, where it transitions to a near-steady level. This is unexpected. The systems being so far dissociated, the error could be characterized as effectively random at that point. Effectively random error grows with sqrt(t), but that growth

isn't visible on this timescale. To check for this behavior, we simulate for a longer time, and plot on a linear scale for better visibility.

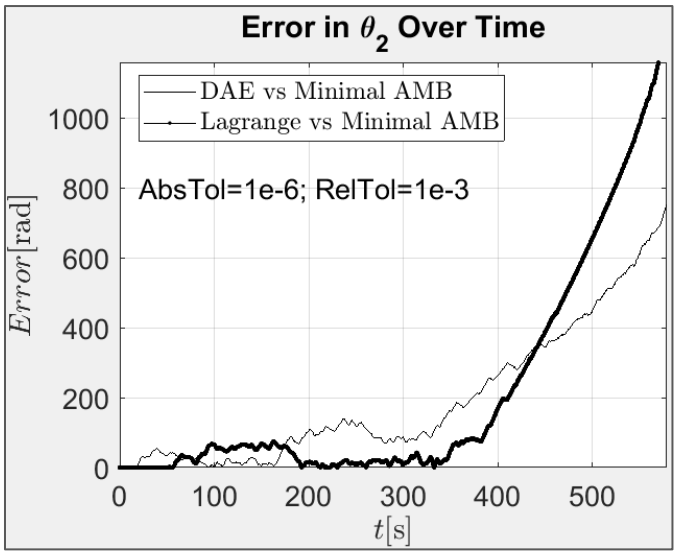


Figure 7: Growth in error over 500 seconds for DAE and Lagrange solutions.

Rather than the expected square root behavior, Figure 7 shows that the error resumes unbounded exponential growth. Reexamination of the hand paths seen in Figure 8 reveal that the numerical integration error slowly adds energy to the system. This creates increasingly fast motion which further increases error and results in unbounded error growth until the ODE solver limits are reached.

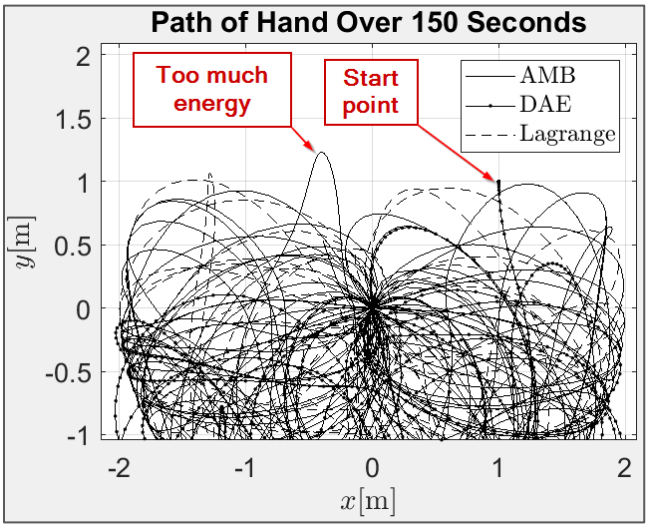


Figure 8: Path of hand over longer period shows trajectory with clearly higher energy than the initial state.

Integration Tolerance Analysis:

Now we investigate the effect of numerical integration tolerances on the growth in error. Simulations are run at 3 different ode45 tolerance combinations, given in Table 1. The "exact"

solution to which the other solutions are compared to calculate error is taken as the most precise trial using the AMB equations of motion. Using these measures, the growth in error in Figure 9 is produced.

ode45 AbsTol	ode45 RelTol
1e-12	1e-9
1e-8	1e-5
1e-4	1e-1

Table 1: Integration tolerances used in plots in Figure 9.

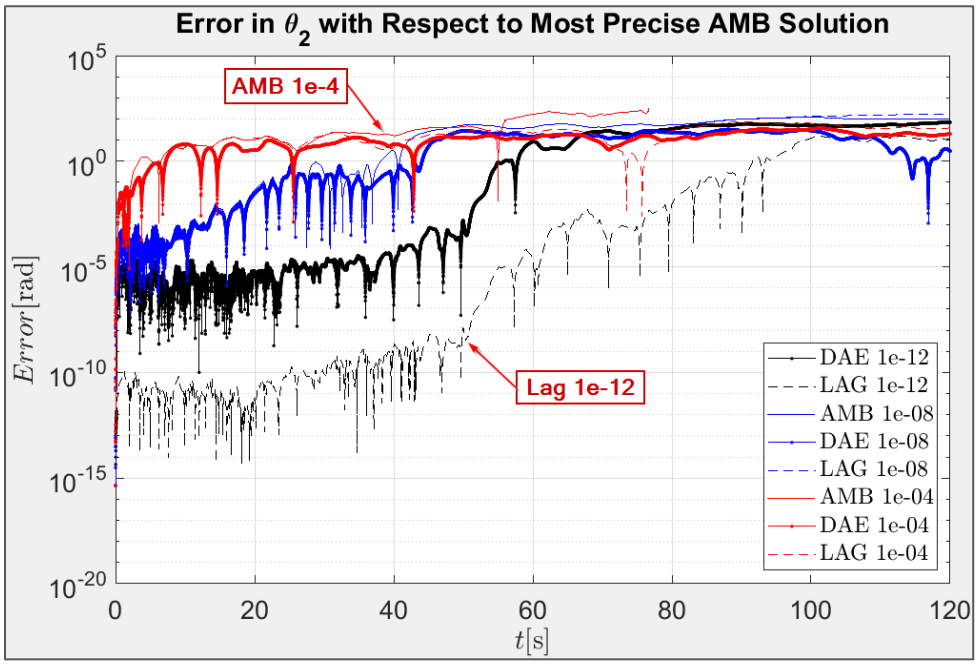


Figure 9: Growth of error for various ode45 tolerances. Number given in the legend is the value of AbsTol.

Figure 9 shows that the tightest-toleranced calculations take the longest to diverge, regardless of the equations of motion. Error growth is still roughly exponential (linear in log plot). The error transition is still near 10 radians for all trials. While the 1e-12 Lagrange trial is clearly more precise than the 1e-12 DAE trial, the difference between Lagrange and DAEs is not so clear for the larger tolerance trials.

The Lagrange and AMB equations of motion are identical, meaning that, theoretically, they produce precisely identical motion and identical integration error. The only cause of divergence is the rounding error for floating-point numbers, which propagates exponentially in the chaotic system. Thus, divergence becomes visible in Figure 9 for the 1e-04 and 1e-08 trials near t = 20 and t = 100 seconds, respectively.

d. Animate your solution.

Appendix: MATLAB Scripts

25. .

a. AMB

i. Derivation

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Assignment: Homework Solutions
Problem:    25a

Description:
This script derives the equations of motion for a double pendulum.
AMB is used for the derivation.
The equations are derived symbolically.
It uses matlabFunction to write a RHS file with the dynamics model.
The CSYS is oriented with x in the direction of gravity
%}

clc; close all;

% Parameters
numLinks = 2;
zeroRow = zeros(1,numLinks);
zeroCol = zeroRow';
syms g real;
L = sym('L', [1 numLinks], 'positive');
m = sym('m', [1 numLinks], 'real');
th = sym('th', [1 numLinks], 'real');
thD = sym('thD', [1 numLinks], 'real');
thDD = sym('thDD', [1 numLinks], 'real');

% Moments of Inertia
I_G = 1/12 * m .* L.^2;

% Unit Vectors
sinTh = sin(th); cosTh = cos(th);
i = [1;0;0];
k = [0;0;1];
lamHat = [cosTh;sinTh;zeroRow];
nHat = [-sinTh;cosTh;zeroRow];

% Position Vectors and Scalar Distances
rG1_O = L(1)/2*lamHat(:,1);   LG1_O = norm(rG1_O);
rE_O = L(1)*lamHat(:,1);      LE_O  = norm(rE_O);
rG2_E = L(2)/2*lamHat(:,2);   LG2_E = norm(rG2_E);
rG2_O = rE_O+rG2_E;           LG2_O = norm(rG2_O);

% Acceleration Vectors
aG1_O = -thD(1)^2 * rG1_O + LG1_O*thDD(1) * nHat(:,1);
aE_O =  -thD(1)^2 * rE_O  + LE_O *thDD(1) * nHat(:,1);
aG2_E = -thD(2)^2 * rG2_E + LG2_E*thDD(2) * nHat(:,2);
aG2_O = aE_O + aG2_E;

% Derivative of Angular Momentum of System
HdotSys_O = sym('Hdot_O', [3 numLinks], 'real');
HdotSys_O(:,1) = m(1)*cross(rG1_O,aG1_O) + I_G(1)*thDD(1)*k;
HdotSys_O(:,2) = m(2)*cross(rG2_O,aG2_O) + I_G(2)*thDD(2)*k;

% Derivative of Angular Momentum of Link 2
Hdot2_P1 = sym('Hdot2_P1', [3 1], 'real');
Hdot2_P1(:,1) = m(2)*cross(rG2_E,aG2_O) + I_G(2)*thDD(2)*k;

% Net Moment Vector on System
Msys_O = m(1)*g*cross(rG1_O,i) + m(2)*g*cross(rG2_O,i);

% Net Moment Vector on Link 2
M2_E = m(2)*g*cross(rG2_E,i);
```

```
% De-vectorize quantities
HdotSys = sum(sum(HdotSys_O));
Msys = sum(Msys_O);
Hdot2 = sum(sum(Hdot2_P1));
M2 = sum(M2_E);

% AMB Equations
eq1 = HdotSys - Msys;
eq2 = Hdot2 - M2;

% Solve
EoM = [eq1; eq2];
M = simplify(jacobian(EoM, thDD));
rhs_stripped = simplify(EoM - M*thDD');
% rhs = simplify(-M\rhs_stripped);

matlabFunction([M, rhs_stripped], 'File', 'Pendulum_2_AMB_Num_Raw');
% matlabFunction(rhs, 'File', 'Pendulum_2_AMB_Raw_REV2');
```

ii. AMB_Num_Raw (matlabFunction from Derivation)

```
function out1 = Pendulum_2_AMB_Num_Raw(L1,L2,g,m1,m2,th1,th2,thD1,thD2)
%PENDULUM_2_AMB_NUM_RAW
%    OUT1 = PENDULUM_2_AMB_NUM_RAW(L1,L2,G,M1,M2,TH1,TH2,THD1,THD2)

%    This function was generated by the Symbolic Math Toolbox version 7.2.
%    19-Oct-2017 23:12:51

t2 = L1.^2;
t3 = th1-th2;
t4 = cos(t3);
t5 = sin(th1);
t6 = sin(t3);
t7 = L1.*L2.*m2.*t4.*(1.0./2.0);
t8 = sin(th2);
t9 = thD1.^2;
out1 =
reshape([t7+m1.*t2.*(1.0./3.0)+m2.*t2,t7,L2.*m2.*(L2.*2.0+L1.*t4.*3.0).*(1.0./6.0),L2.^2.*m2.*(1.
0./3.0),L1.*g.*m1.*t5.*(1.0./2.0)+L1.*g.*m2.*t5+L2.*g.*m2.*t8.*(1.0./2.0)-
L1.*L2.*m2.*t6.*t9.*(1.0./2.0)+L1.*L2.*m2.*t6.*thD2.^2.*(1.0./2.0),L2.*m2.*(g.*t8.*2.0-
L1.*t6.*t9.*2.0).*(1.0./4.0)],[2,3]);
```

iii. AMB_Num

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Assignment: Homework Solutions
Problem:    25a

Description:
This function models the dynamics of a double pendulum from DAEs.
It formats the arguments and calls the auto-generated
Pendulum_2_DAE_Raw.
It assembles the outputs to return the full state derivative.
%}

function res = Pendulum_2_AMB_Num(t,Z0,p)

% Unpack Parameters
L1 = p.L(1); L2 = p.L(2);
m1 = p.m(1); m2 = p.m(2);
g = p.g;

% Unpack State Variables
th1 = Z0(1); th2 = Z0(2); thD1 = Z0(3); thD2 = Z0(4);

% Call function to numerically calculate accelerations
[numMat] = Pendulum_2_AMB_Num_Raw(L1,L2,g,m1,m2,th1,th2,thD1,thD2);

% Post-process output to extract mass matrix and rhs_stripped vector
rhs_stripped = numMat(:,end);
```

```
M = numMat(:,1:end-1);

% Calculate accelerations
rhs = -M\rhs_stripped;

res = [thD1; thD2; rhs];
```

b. DAE

i. Derivation

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Assignment: Homework Solutions
Problem:    25b

Description:
This script derives the equations of motion for a double pendulum.
DAEs in Cartesian coordinates are used for the derivation.
The DAEs have 6 DOF with 4 constraint equations to form a 10x10 matrix
The equations are derived symbolically.
It uses matlabFunction to write a RHS file with the dynamics model.
The CSYS is oriented with x in the direction of gravity
%}

clc; close all;

% Parameters
numLinks = 2;
numDim = 3;
zeroRow = zeros(1,numLinks);
zeroCol = zeroRow';
syms g real;
L = sym('L', [1 numLinks], 'real');
m = sym('m', [1 numLinks], 'real');
x = sym('x', [1 numLinks], 'real'); %x for points [G1, G2]
y = sym('y', [1 numLinks], 'real'); %y for points [G1, G2]
th = sym('th', [1 numLinks], 'real'); %theta for objects [1 2]
xD = sym('xD', [1 numLinks], 'real'); %xDot for points [G1, G2]
yD = sym('yD', [1 numLinks], 'real'); %yDot for points [G1, G2]
thD = sym('thD', [1 numLinks], 'real'); %thetaDot for objects [1 2]
xDD = sym('xDD', [1 numLinks], 'real'); %xDoubleDot for points [G1, G2]
yDD = sym('yDD', [1 numLinks], 'real'); %yDoubleDot for points [G1, G2]
thDD = sym('thDD', [1 numLinks], 'real'); %thetaDoubleDot for objects [1 2]
syms FOx FOy FEx FEy real

% Position Vectors
rG1_O = [x(1); y(1)];
rE_O = 2*rG1_O;
rG2_O = [x(2); y(2)];
rG2_E = rG2_O - rE_O;

% Moments of Inertia
I_G = 1/12 * m .* L.^2;

% Trig Functions
sinTh = sin(th); cosTh = cos(th);

% Link 1 LMB, AMB
F1x = FOx - FEx + m(1)*g;
F1y = FOy - FEy;
M_G(1) = L(1)/2 * (FOx*sinTh(1) - FOy*cosTh(1) + FEx*sinTh(1) - FEy*cosTh(1));

% Link 2 LMB, AMB
F2x = FEx + m(2)*g;
F2y = FEy;
M_G(2) = L(2)/2 * (FEx*sinTh(2) - FEy*cosTh(2));

% Link Momentum Balance Equations
FNetx = [F1x; F2x];
FNety = [F1y; F2y];
```

```
eq(1:2) = (FNetx./m') - xDD';
eq(3:4) = (FNety./m') - yDD';
eq(5:6) = (M_G./I_G) - thDD;

% Constraint Equations
% x(1)*xDD(1) + xD(1)^2 + y(1)*yDD(1) + yD(1)^2; %Equation
eq(7:8) = -xDD - L/2.*(thD.^2.*cosTh + thDD.*sinTh);
eq(8) = eq(8) + 2*xDD(1);
eq(9:10) = -yDD + L/2.*(-thD.^2.*sinTh + thDD.*cosTh);
eq(10) = eq(10) + 2*yDD(1);

% Solve equation
stateVars = [xDD, yDD, thDD, FOx, FOy, FEx, FEy];
M = jacobian(eq, stateVars);
rhs_stripped = simplify(eq' - M*stateVars');
% rhs = simplify(-M\rhs_stripped);

matlabFunction([M, rhs_stripped], 'File', 'Pendulum_2_DAE_Num_Raw');
```

ii. DAE_Num_Raw (matlabFunction from Derivation)

```
function out1 = Pendulum_2_DAE_Num_Raw(L1,L2,g,m1,m2,th1,th2,thD1,thD2)
%PENDULUM_2_DAE_NUM_RAW
%    OUT1 = PENDULUM_2_DAE_NUM_RAW(L1,L2,G,M1,M2,TH1,TH2,THD1,THD2)

%    This function was generated by the Symbolic Math Toolbox version 7.2.
%    19-Oct-2017 22:50:15

t2 = 1.0./m1;
t3 = 1.0./m2;
t4 = 1.0./L1;
t5 = sin(th1);
t6 = t2.*t4.*t5.*6.0;
t7 = cos(th1);
t8 = 1.0./L2;
t9 = sin(th2);
t10 = cos(th2);
t11 = thD1.^2;
t12 = thD2.^2;
out1 = reshape([-1.0,0.0,0.0,0.0,0.0,0.0,-1.0,2.0,0.0,0.0,0.0,-1.0,0.0,0.0,0.0,0.0,0.0,-
1.0,0.0,0.0,0.0,0.0,-1.0,0.0,0.0,0.0,0.0,0.0,-1.0,2.0,0.0,0.0,0.0,-1.0,0.0,0.0,0.0,0.0,0.0,-
1.0,0.0,0.0,0.0,0.0,-1.0,0.0,L1.*t5.*(-1.0./2.0),0.0,L1.*t7.*(1.0./2.0),0.0,0.0,0.0,0.0,0.0,0.0,-
1.0,0.0,L2.*t9.*(-
1.0./2.0),0.0,L2.*t10.*(1.0./2.0),t2,0.0,0.0,0.0,t6,0.0,0.0,0.0,0.0,0.0,0.0,0.0,t2,0.0,t2.*t4.*t7
.*-6.0,0.0,0.0,0.0,0.0,0.0,-t2,t3,0.0,0.0,t6,t3.*t8.*t9.*6.0,0.0,0.0,0.0,0.0,0.0,0.0,-
t2,t3,t2.*t4.*t7.*-6.0,t3.*t8.*t10.*-6.0,0.0,0.0,0.0,0.0,0.0,g,g,0.0,0.0,0.0,0.0,L1.*t7.*t11.*(-
1.0./2.0),L2.*t10.*t12.*(-1.0./2.0),L1.*t5.*t11.*(-1.0./2.0),L2.*t9.*t12.*(-1.0./2.0)],[10,11]);
```

iii. DAE_Num

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Assignment: Homework Solutions
Problem:    25b

Description:
This function models the dynamics of a double pendulum from DAEs.
It formats the arguments and calls the auto-generated
Pendulum_2_DAE_Raw.
It assembles the outputs to return the full state derivative.
%}

function res = Pendulum_2_DAE_Num(t,Z0,p)

% Unpack Parameters
L1 = p.L(1); L2 = p.L(2);
m1 = p.m(1); m2 = p.m(2);
g = p.g;

% Unpack State Variables
x1 = Z0(1);     x2 = Z0(2);
y1 = Z0(3);     y2 = Z0(4);
```

```
th1 = Z0(5);    th2 = Z0(6);
xD1 = Z0(7);    xD2 = Z0(8);
yD1 = Z0(9);    yD2 = Z0(10);
thD1 = Z0(11);  thD2 = Z0(12);

% Call function to numerically calculate accelerations
[numMat] = Pendulum_2_DAE_Num_Raw(L1,L2,g,m1,m2,th1,th2,thD1,thD2);

% Post-process output to extract mass matrix and rhs_stripped vector
rhs_stripped = numMat(:,end);
M = numMat(:,1:end-1);

% Calculate accelerations and constraint forces
rhs = -M\rhs_stripped;

res = [xD1; xD2; yD1; yD2; thD1; thD2; rhs(1:6)];
```

c. Lagrange

i. Derivation

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Assignment: Homework Solutions
Problem:    25c

Description:
This script derives the equations of motion for a double pendulum.
Lagrange equations are used for the derivation.
The equations are derived symbolically.
It uses matlabFunction to write a RHS file with the equations of motion.
The CSYS is oriented with x in the direction of gravity
%}

clc; close all;

% Parameters
numLinks = 2;
numDim = 3;
zeroRow = zeros(1,numLinks);
zeroCol = zeroRow';
syms g real;
L = sym('L', [1 numLinks], 'positive');
m = sym('m', [1 numLinks], 'real');
th = sym('th', [1 numLinks], 'real');
thD = sym('thD', [1 numLinks], 'real');
thDD = sym('thDD', [1 numLinks], 'real');

% Moments of Inertia
I_G = 1/12 * m .* L.^2;

% Unit Vectors
sinTh = sin(th); cosTh = cos(th);
i = [1;0;0]; k = [0;0;1];
lamHat = [cosTh;sinTh;zeroRow];

% Position Vectors
rG1_O = L(1)/2*lamHat(:,1);     LG1_O = norm(rG1_O);
rE_O = 2 * rG1_O;               LE_O  = norm(rE_O);
rG2_E = L(2)/2*lamHat(:,2);     LG2_E = norm(rG2_E);
rG2_O = rE_O+rG2_E;             LG2_O = norm(rG2_O);

% Velocity Vectors
vG1_O =  thD(1)*cross(k,rG1_O);
vE_O = 2 * vG1_O;
vG2_E = thD(2)*cross(k,rG2_E);
vG2_O = vG2_E + vE_O;

% Energy Expressions
KETrans = 1/2*m(1)*(vG1_O'*vG1_O) + 1/2*m(2)*(vG2_O'*vG2_O);
KERot =   sum(1/2*I_G.*thD.^2);
```

```
KE = KETrans + KERot;
xG = [dot(rG1_O, i), dot(rG2_O, i)];
PE = sum(-m*g.*xG);

% Lagrange Quantities
Lag = KE - PE;
q = th; qDot = thD; qDDot = thDD;
% dLdq = jacobian(Lag, q);
% dLdqDot = jacobian(Lag, qDot);
% ddt_dLdqDot = jacobian(dLdqDot, [q qDot])*[qDot, qDDot]';

% Lagrange Equations
EoM = jacobian(jacobian(Lag, qDot), [q qDot])*[qDot, qDDot]' - jacobian(Lag, q)';

% Solution for thetaDoubleDot
M = jacobian(EoM, thDD);
rhs_stripped = simplify(EoM - M*qDDot');
% rhs = -M\rhs_stripped;

% matlabFunction(rhs, 'File', 'Pendulum_2_Lag_Raw');
matlabFunction([M, rhs_stripped], 'File', 'Pendulum_2_Lag_Num_Raw');
```

ii. Lag_Num_Raw (matlabFunction from Derivation)

```
function out1 = Pendulum_2_Lag_Num_Raw(L1,L2,g,m1,m2,th1,th2,thD1,thD2)
%PENDULUM_2_LAG_NUM_RAW
%    OUT1 = PENDULUM_2_LAG_NUM_RAW(L1,L2,G,M1,M2,TH1,TH2,THD1,THD2)

%    This function was generated by the Symbolic Math Toolbox version 7.2.
%    19-Oct-2017 23:29:55

t2 = L1.^2;
t3 = cos(th1);
t4 = sin(th1);
t5 = t3.^2;
t6 = t4.^2;
t7 = cos(th2);
t8 = L1.*L2.*t3.*t7;
t9 = sin(th2);
t10 = L1.*L2.*t4.*t9;
t11 = t8+t10;
t12 = m2.*t11.*(1.0./2.0);
t13 = L2.^2;
t14 = th1-th2;
t15 = sin(t14);
out1 =
reshape([m1.*t2.*(1.0./1.2e1)+m2.*(t2.*t5.*2.0+t2.*t6.*2.0).*(1.0./2.0)+m1.*(t2.*t5.*(1.0./2.0)+t
2.*t6.*(1.0./2.0)).*(1.0./2.0),t12,t12,m2.*t13.*(1.0./1.2e1)+m2.*(t7.^2.*t13.*(1.0./2.0)+t9.^2.*t
13.*(1.0./2.0)).*(1.0./2.0),L1.*(g.*m1.*t4+g.*m2.*t4.*2.0+L2.*m2.*t15.*thD2.^2).*(1.0./2.0),L2.*m
2.*(g.*t9-L1.*t15.*thD1.^2).*(1.0./2.0)],[2,3]);
```

iii. Lag_Num

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Assignment: Homework Solutions
Problem:    25c

Description:
This function models the dynamics of a double pendulum from DAEs.
It formats the arguments and calls the auto-generated
Pendulum_2_DAE_Raw.
It assembles the outputs to return the full state derivative.
%}

function res = Pendulum_2_Lag_Num(t,Z0,p)

% Unpack Parameters
L1 = p.L(1); L2 = p.L(2);
m1 = p.m(1); m2 = p.m(2);
g = p.g;
```

```
% Unpack State Variables
th1 = Z0(1); th2 = Z0(2); thD1 = Z0(3); thD2 = Z0(4);

% Call function to numerically calculate accelerations
[numMat] = Pendulum_2_Lag_Num_Raw(L1,L2,g,m1,m2,th1,th2,thD1,thD2);

% Post-process output to extract mass matrix and rhs_stripped vector
rhs_stripped = numMat(:,end);
M = numMat(:,1:end-1);

% Calculate accelerations
rhs = -M\rhs_stripped;

res = [thD1; thD2; rhs];
```

d. Other files

i. ROOT

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Problem:    25: Double Pendulum

Description:
This script tests the RHS files created for a double pendulum.
The RHS files tested use AMB, DAEs, and Lagrange equations
It plots the paths of the end of the forearm for each solution method.
%}

clc; close all;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% INPUTS %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Results selection
animate = 1; %      0: hide animations 1: show animations
plotPaths = 1;%     0: hide paths      1: show path plots
plotErrors = 1;%    0: hide errors     1: error log plot   2: error linear plot

% Parameters
tFinal = 30; %Simulation time
L = 1;
m = 1;
p.g = 1; %Gravitational acceleration
p.m = [m m]; %Link masses
p.L = [L L]; %Full link lengths

% Initial Conditions
thInit = deg2rad([90 180]); %Measured from +x, the direction of gravity
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% INPUTS %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Initial Conditions
xInit = 0.5*p.L.*cos(thInit);
xInit(2) = xInit(2) + 2*xInit(1);
yInit = 0.5*p.L.*sin(thInit);
yInit(2) = yInit(2) + 2*yInit(1);
ratesInit = zeros(1,6); %System is initially motionless
z0 = [thInit, 0, 0]; %State Vector [theta1, theta2, thetaDot1, thetaDot2]
z0DAE = [xInit, yInit, thInit, ratesInit]; % State Vector for DAEs
% [x1 x2 y1 y2 theta1 theta2 x1Dot x2Dot y1Dot y2Dot theta1Dot theta2Dot]

% Time & Output Arrays
tSpan = [0, tFinal];

% Solve ODEs
% AMB
[tAMB, ZAMB] = ode45(@(t,Z) Pendulum_2_AMB_Num(t,Z,p), tSpan, z0);
% DAEs
[tDAE, ZDAE] = ode45(@(t,Z) Pendulum_2_DAE_Num(t,Z,p), tSpan, z0DAE);
% Lagrange
[tLag, ZLag] = ode45(@(t,Z) Pendulum_2_Lag_Num(t,Z,p), tSpan, z0);

% Post-process to find x(t), y(t) of the link endpoints
```

```
[xAMB, yAMB] = Pendulum_PostProcess(    ZAMB(:,1:2),p);
[xDAE, yDAE] = Pendulum_PostProcess_DAE(ZDAE(:,1:2),ZDAE(:,3:4),ZDAE(:,5:6),p);
[xLag, yLag] = Pendulum_PostProcess(    ZLag(:,1:2),p);

% Assign Path Plotting Variables
sizes = [length(tAMB), length(tDAE), length(tLag)];
maxLen = max(sizes);
plotX = zeros(maxLen,3);
plotY = zeros(maxLen,3);
plotX(1:sizes(1),1) = yAMB(:,3);
plotX(1:sizes(2),2) = yDAE(:,3);
plotX(1:sizes(3),3) = yLag(:,3);
plotY(1:sizes(1),1) = -xAMB(:,3);
plotY(1:sizes(2),2) = -xDAE(:,3);
plotY(1:sizes(3),3) = -xLag(:,3);

% Plot Forearm End Paths
if (plotPaths)
    lineStyles = {'k', 'k.-', 'k--'};
    lineNames = {'AMB', 'DAE', 'Lagrange'};
    pathFig = figure;
    for i = [1,2,3]
        plot(plotX(1:sizes(i),i), plotY(1:sizes(i),i), lineStyles{i},...
            'DisplayName', lineNames{i});
        hold on;
    end
    titleStr = sprintf('Path of Hand Over %i Seconds', tFinal);
    title(titleStr);
    xlabel('$x \mathrm{[m]}$', 'Interpreter', 'latex');
    ylabel('$y \mathrm{[m]}$', 'Interpreter', 'latex');
    set(gca, 'FontSize', 16);
    leg = legend('show'); leg.Interpreter = 'latex';
    grid on; axis equal;
end


% Interpolate theta vectors
thAMB = ZAMB(:,2); thDAE = ZDAE(:,6); thLag = ZLag(:,2);
% thetaDAEAMBTime is thDAE with the values interpolated to the values in
% the tAMB time vector. This allows direct comparison to thAMB
thetaDAEAMBTime = interp1(tDAE, thDAE, tAMB);
thetaLagAMBTime = interp1(tLag, thLag, tAMB);

% Assign Path Plotting Variables
errorDAE = abs(thAMB - thetaDAEAMBTime);
errorLag = abs(thAMB - thetaLagAMBTime);
plotX = tAMB;
plotY = [errorDAE, errorLag];

if (plotErrors)
% Plot Error Growth
    lineStyles = {'k', 'k.-', 'k--'};
    lineNames = {'DAE vs Minimal AMB', 'Lagrange vs Minimal AMB'};
    errorFig = figure;
    for i = 1:2
        if(plotErrors == 1)
            semilogy(plotX, plotY(:,i), lineStyles{i},...
                'DisplayName', lineNames{i});
        else
            plot(plotX, plotY(:,i), lineStyles{i},...
                'DisplayName', lineNames{i});
        end
        hold on;
    end
    title('Error in \theta_2 Over Time');
    xlabel('$t\mathrm{[s]}$', 'Interpreter', 'latex');
    ylabel('$Error\mathrm{[rad]}$', 'Interpreter', 'latex');
    set(gca, 'FontSize', 16);
    leg = legend('show'); leg.Interpreter = 'latex';
    grid on;
end
```

```
if(animate)
    % Animation AMB
    tAnim = 10;
    animX = yAMB;
    animY = -xAMB;
    Animate_Links(tAMB, animX, animY, tAnim);

    % Animation DAE
    animX = yDAE;
    animY = -xDAE;
    Animate_Links(tDAE, animX, animY, tAnim);

    % Animation AMB
    animX = yLag;
    animY = -xLag;
    Animate_Links(tLag, animX, animY, tAnim);
end
```

ii. ROOT_ToleranceVar

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Problem:    25: Double Pendulum

Description:
This script tests the RHS files created for a double pendulum.
The RHS files tested use AMB, DAEs, and Lagrange equations.
It calculates the paths using varying ode45 tolerances.
It plots the errorr in theta2 with respect to the most precise AMB trial.
%}

clc; close all;
clear plotY; clear errorTh; clear lineNames;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% INPUTS %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Parameters
tFinal = 120; %Simulation time
L = 1;
m = 1;
p.g = 1; %Gravitational acceleration
p.m = [m m]; %Link masses
p.L = [L L]; %Full link length

% Initial Conditions
thInit = deg2rad([90 180]); %Measured from +x, the direction of gravity

% Tolerances
tolMin = 1e-12; %Min abs tolerance
tolMax = 1e-4; %Max abs tolerance
relMult = 1000; %Multiplier: reltol = absTol * relMult
numTol = 3; %# of trials to try with different tolerances
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% INPUTS %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


% Initial Conditions
xInit = 0.5*p.L.*cos(thInit);
xInit(2) = xInit(2) + 2*xInit(1);
yInit = 0.5*p.L.*sin(thInit);
yInit(2) = yInit(2) + 2*yInit(1);
ratesInit = zeros(1,6); %System is initially motionless
z0 = [thInit, 0, 0]; %State Vector [theta1, theta2, thetaDot1, thetaDot2]
z0DAE = [xInit, yInit, thInit, ratesInit]; % State Vector for DAEs
% [x1 x2 y1 y2 theta1 theta2 x1Dot x2Dot y1Dot y2Dot theta1Dot theta2Dot]
tSpan = [0 tFinal];

% Output Arrays
tMaster = [];
thMaster = [];
```

```
lineNames = {};
errorTh = [];

% ode45 Tolerances
% absTols, relTols: log-spaced array with tolerances for all trials
absTols = logspace(log(tolMin)/log(10),...
    log(tolMax)/log(10), numTol);
relTols = relMult * logspace(log(tolMin)/log(10),...
    log(tolMax)/log(10), numTol);

% Solve ODEs
for i=1:numTol
    isMaster = 0; %Assume this trial is not the master for error calculation

    % Solve ODE
    opts = odeset('RelTol', relTols(i), 'AbsTol', absTols(i));
        % AMB
    [tAMB, ZAMB] = ode45(@(t,Z) Pendulum_2_AMB_Num(t,Z,p), tSpan, z0, opts);
        % DAEs
    [tDAE, ZDAE] = ode45(@(t,Z) Pendulum_2_DAE_Num(t,Z,p), tSpan, z0DAE, opts);
        % Lagrange
    [tLag, ZLag] = ode45(@(t,Z) Pendulum_2_Lag_Num(t,Z,p), tSpan, z0, opts);

    % Assign master values for calculating errors to most precise AMB trial
    if(i == 1)
        isMaster = 1;
        tMaster = tAMB;
        thMaster = ZAMB(:,2);
    end

    % Interpolate theta vectors
    thAMB = ZAMB(:,2); thDAE = ZDAE(:,6); thLag = ZLag(:,2);
    % thetaDAEAMBTime is thDAE with the values interpolated to the values in
    % the tAMB time vector. This allows direct comparison to thAMB
    thetaAMBMasterTime = interp1(tAMB, thAMB, tMaster);
    thetaDAEMasterTime = interp1(tDAE, thDAE, tMaster);
    thetaLagMasterTime = interp1(tLag, thLag, tMaster);

    % Calculate errors in theta2
    errorAMB = abs(thMaster - thetaAMBMasterTime);
    errorDAE = abs(thMaster - thetaDAEMasterTime);
    errorLag = abs(thMaster - thetaLagMasterTime);

    if(~isMaster) %For lower-tolerance AMB trials
        errorTh = [errorTh, errorAMB];
        name = sprintf('AMB %.0d', absTols(i));
        lineNames(length(lineNames)+1) = {name};
    end

    errorTh = [errorTh, errorDAE, errorLag];
    name1 = sprintf('DAE %.0d', absTols(i));
    name2 = sprintf('LAG %.0d', absTols(i));
    newInd = length(lineNames)+1;
    lineNames(newInd:newInd+1) = {name1, name2};
end

% Plot Error Growth
plotX = tMaster;
plotY = errorTh;

lineStyles = {'k.-', 'k--', 'b', 'b.-', 'b--','r', 'r.-', 'r--',...
    'g', 'g.-', 'g--', 'c', 'c.-', 'c--'};
%If plotting >5 tolerances, add more items to the lineStyles cell.
errorFig = figure;
for i = 1:length(plotY(1,:))
    semilogy(plotX, plotY(:,i), lineStyles{i},...
        'DisplayName', lineNames{i});
    hold on;
end
title('Error in \theta_2 with Respect to Most Precise AMB Solution');
xlabel('$t\mathrm{[s]}$', 'Interpreter', 'latex');
```

```
ylabel('$Error\mathrm{[rad]}$', 'Interpreter', 'latex');
set(gca, 'FontSize', 16);
leg = legend('show'); leg.Interpreter = 'latex';
grid on;
```

iii. Pendulum_Postprocess

```
%{
Name:       Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Created: 2017-10-09
Updated: 2017-10-20

Description:
This function accepts a time and theta vector for an n-linked pendulum.
It computes the Cartesian coordinate position vectors of the pivots
It returns the list of coordinates, plus the first pivot at 0,0.

Output format:  [xBase, xTip, yBase, yTip]
%}

function [x, y] = Pendulum_PostProcess(th, p)

timePts = length(th(:,1)); % Number of time points
n = length(th(1,:)); % Number of links
L = p.L;

% Compute coordinates
x = zeros(timePts,n+1); y = zeros(timePts,n+1);
x(:,2) = L(1)*cos(th(:,1)); %First endpoint (elbow) x coordinates
y(:,2) = L(1)*sin(th(:,1)); %First endpoint (elbow) y coordinates
for i = 2:n
%     Next endpoint calculation starts from previous endpoint
    x(:,i+1) = x(:,i) + L(i)*cos(th(:,i));
    y(:,i+1) = y(:,i) + L(i)*sin(th(:,i));
end

end
```

iv. Pendulum_Postprocess_DAE

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Created:    2017-10-19
Updated:    2017-10-19

Description:
This function accepts DAE solver outputs for an n-link pendulum.
The inputs xCG, yCG, th are mxn matrices composed of column vectors, where:
    m = number of time points
    n = number of links in pendulum
m and n must be equal for each matrix input.
It computes the Cartesian coordinate position vectors of the endpoints.
The coordinate system is oriented with +x in the direction of gravity.
Return:     List of endpoint (x,y) coordinates, including the shoulder.

Input format:   [xCG_1, xCG_2],     [yCG_1, yCG_2],     [theta_1, theta_2]
Output format:  [xShoulder, xElbow, xHand, yShoulder, yElbow, yHand]
%}

function [x, y] = Pendulum_PostProcess_DAE(xCG, yCG, th, p)

n = length(xCG(1,:)); %Number of links

% Unpack parameters
timePts = length(xCG(:,1)); %Number of time points
halfL = p.L/2; %Lengths from CG to end of link

% Compute coordinates
x = zeros(timePts,n+1); y = zeros(timePts,n+1);
x(:,2:end) = xCG + halfL.*cos(th);
y(:,2:end) = yCG + halfL.*sin(th);
```

```
end
```

v. Animate_Links

```
%{
Author:     Aaron Sandoval
Course:     MAE 5730 - Intermediate Dynamics and Vibrations
Created: 2017-10-19
Updated: 2017-10-20

Description:
This function animates the motion of an arbitrary numbe of links.
It accepts a series of
%}

function  Animate_Links(t, x, y, tAnim)

% Setup variables
numP = length(x(1,:));
numL = numP-1;
xyMax = 1.1*max(max(max(abs(x), max(abs(y))))); %Yes, all those are necessary
xMax = max(max(x));
xMin = min(min(x));
yMax = max(max(y));
yMin = min(min(y));
eps = .01*xyMax;

% Define shape geometry centered on origin
% shape= [
%     eps     eps     -eps    -eps    +eps
%     -eps    +eps    +eps    -eps    -eps
%        ];

% plotX = shape(1,:);
% plotY = shape(2,:);

figure;
colors = {'r', 'g', 'b', 'k', 'o'};
for i = 1:numL
    plt(i) = plot(x(1,i:i+1),y(1,i:i+1), 'LineWidth', 5,...
        'Color', colors{i});
    hold on;
end
plotRange = [xMax-xMin, yMax-yMin];
margin = .05*plotRange+2*eps;
axis('equal');
axis([xMin-margin(1) xMax+margin(1) yMin-margin(2) yMax+margin(2)]);
set(gca, 'FontSize', 16);
% grid on;

tNorm = t*(tAnim/t(length(t)));
tic;
shg
cur = toc;
while cur<tAnim
    for i=1:numP
        xNew(i) = interp1(tNorm, x(:,i), cur);
        yNew(i) = interp1(tNorm, y(:,i), cur);
%         zNew = [x+xNew; y+yNew];
        if(i>1)
            plt(i-1).XData = xNew(i-1:i);
            plt(i-1).YData = yNew(i-1:i);
        end
    end
    drawnow;
    cur = toc;
end
end
```